

电商搜索排序系统设计：一次完整的推演记录

题源：Coupang L6 Staff 面经中出现的原题。它在面试中的实际形式是「从你简历上的项目出发，重新设计一遍」——一位候选人的记录写着「没想到会考这个」。

约束：P99 端到端 200ms，峰值 50K QPS，商品目录 5 亿。

面试官背景是后端，不做 ML。 因此本文的一条隐含规则：模型内部压成一个盒子，重点放在它周围的后端问题——召回、特征服务、索引更新、扇出、尾延迟、成本。

1. 整体链路

```
query
  → NLP (tokenize / 拼写纠正 / query feature)
  → top aggregator
  → 扇出到全部 index server
  → 检索 (倒排)
  → 单机轻量排序, 返回 top N
  → top aggregator 汇总, 重排
  → 返回 doc_id 列表
```

最后一步值得单独说：**返回给前端的只有 doc_id**。商品的完整字段由前端向另一套商品服务获取。这让索引保持精简，也把展示层的读压力挪出了排序系统。

2. 索引的三个部分

结构	内容	更新特性
term → doc_id list	倒排索引	改动代价高（要改 posting list），但变更率极低
doc_id → meta	标题、常用字段	中等
doc_id → feature	价格、库存、点击数、评分、embedding	点更新，便宜，变更率最高

分片键：doc_id hash，100 个分片。

这个选择有一个重要推论：**同一个 doc 的三份数据都在同一台机器上**。因此重排所需的特征不需要远程调用，检索完直接本地读——这是把特征服务这一跳省掉的关键。

索引只存标题和常用字段，5 亿商品约 5TB，每台 50GB。

3. 单机检索与排序的延迟拆解

一次查询在单个 index server 上的处理：

阶段	做什么	耗时
检索	倒排求交，命中约 10 万候选	1ms
粗排	线性打分 (token 匹配得分)， 桶排序 取 top 1000	2ms
精排	GBDT 每条 0.5ms，对 top 40 打分，选出 25 条	20ms
合计		22ms

桶排序这一步值得单独说明：把浮点分值映射到 `[0, 65535]` 的整数区间，用计数排序取 top-K，复杂度 $O(n)$ 而非 $O(n \log n)$ 。10 万候选 2ms 出结果。这类「用值域有界换掉比较排序」的技巧在检索系统里很常见，是可以主动提出来的加分点。

单机吞吐： $64 \text{ 核} \div 22\text{ms} \approx \textbf{3K QPS}$ 。

4. 重排层

top aggregator 汇总各分片结果，取 top 500，跑重型 DNN：

每条 2ms × 500 条	= 1000 核·ms
64 核并行 → 墙钟	= 16ms
单机吞吐 = 1000ms / 16ms	= 62.5 QPS

端到端延迟： $22 \text{ (检索)} + 16 \text{ (重排)} + \text{NLP 约 } 2\text{ms} \approx \textbf{40ms}$ ，相对 200ms 的 P99 预算有 5 倍富余。

5. Embedding 与特征

离线算好的：- doc embedding —— 随索引一起存在 index server 上

在线算的：- query embedding、user embedding —— 各 1-2ms，与 NLP 并行，不额外增加延迟

实时信号（价格、库存、促销、点击数、评分）：不走独立的特征服务，而是随 `doc_id → feature` 存在 index server 本地。这是本设计的关键取舍：

- **收益：**重排时零远程调用，省掉一整跳的延迟与一个可能被打爆的组件
- **代价：**变更必须推送到该分片的**全部副本**，且要接受一个一致性窗口

6. 索引更新：三条线

线	承载内容	量级	机制
实时	价格、库存、促销、点击计数	最大	KV 点更新
增量批	新品上架、下架、标题描述编辑、评论文本	小	增量段 + 定期合并
全量重建	索引结构变更、数据修复	低频	离线重建后切换

6.1 实时线的量

关键点：每条变更要写到该分片的所有副本，副本数不摊薄写入。因此除数是分片数，不是机器数。

变更率假设	全局 QPS	每分片（100 分片）
1% / 秒	500 万	5 万/秒
0.1% / 秒	50 万	5000/秒
1% / 分钟	8.3 万	833/秒

取 0.1%/秒：**每台 5000 写入/秒**，内存 KV 轻载。需要说明的是写不能拖慢读的 P99——批量 apply、或读写分离到不同线程。

顺带：1%/秒 意味着整个目录每 100 秒全量翻新一次，这个假设本身不成立。**在系统设计中，先检查假设是否荒谬，往往比优化方案更有效。**

6.2 互动信号的量

按 5000 万 DAU 估：

信号	频次/人/天	全局 QPS
商品浏览/点击	30	17,400
加购	2	1,160
收藏	1	580
购买	0.3	174
评论	0.02	12
合计		约 1.9 万/秒（峰值 5 倍 $\approx$ 10 万/秒）

分到 100 分片：均值每台 193/秒，峰值 964/秒——比价格库存那条低一个数量级。原因是这些事件天然按 doc 分散。

优化： 点击计数是聚合值，不需要实时精确。流式聚合（几秒一个窗口）后批量更新，可以把 100 次点击合并成一次写。

6.3 倒排索引的真实变更率

只有 term 被增删时才动倒排：

事件	频次	QPS
新商品上架	20 万/天	2.3
商品下架	15 万/天	1.7
标题/描述编辑	200 万/天	23
新评论文本	100 万/天	12
合计		约 39/秒

机内扇出： 一条评论包含约 100 个不同 term，就要更新 100 个 posting list。但由于按 doc_id 分片，这 100 次写全部落在同一台机器上，不跨机。

全局 $39 \times 100 = 3900$ 次 posting list 写/秒；分散到 100 个分片，每台约 39/秒。

与 doc-feature 的 5000/秒 相差两个数量级。这解释了为什么三条更新线必须分开：倒排改动贵但极少，feature 改动便宜但频繁。

热点提醒： 爆款商品短时间涌入大量评论时，那些写全部落在同一台机器上。每秒几十条评论 $\times$ 100 term = 几千次 posting list 写，单机仍可承受，但值得主动提出。

7. 缓存

电商 query 分布是极端的头部集中 + 长尾。查询结果缓存命中率按 80% 估：

50K QPS 峰值 $\rightarrow$ 后端实际 10K QPS

这是整个设计中最便宜的优化，直接把机器数砍到五分之一。

8. 机器数与成本

Index 层

每次查询扇出到全部 100 个分片，所以每个分片都要承受完整的后端 QPS：

每分片需承受 10K QPS ÷ 单机 3K QPS = 4 副本
× 2 (峰值与利用率余量) = 8 副本
100 分片 × 8 = 800 台

重排层

10K QPS ÷ 62.5 QPS/台 = 160 台
× 2 = 320 台

合计约 1120 台。

单价：64 核 + 128-256GB 内存（索引需常驻）+ NVMe，约 1.5-2 万美元。硬件总价约 **1700-2200 万美元**；加机架、电力、冷却、网络与运维，三年 TCO 通常是硬件价的 2-3 倍。

常见错误：只按分片数算机器（漏掉副本），或只算一层（漏掉重排层）。两者都会让成本估算差一个量级。

9. 尾延迟：三个闭式结论

均值 40ms 对 200ms 的 P99 预算看似宽裕，但扇出结构会放大尾巴。以下三个关系是可以推导的，不需要压测。

9.1 扇出把百分位要求推高 N 倍量级

N 个分片独立，每个在时间 t 内完成的概率为 p，则全部完成的概率为 p^N 。

要保证整体 P99： $p \geq 0.99^{(1/N)}$

分片数 N	每个分片需达到
10	P99.90
50	P99.98
100	P99.99
1000	P99.999

这就是「扇出 100 台要求单分片 P99.99」的来源。

9.2 利用率决定尾巴，且收益递减

M/M/1 的逗留时间服从指数分布，因此 $T(x) / T_{\text{mean}} = -\ln(1-x)$ ：

- $P95 = 3.0 \times \text{均值}$
- **$P99 = 4.6 \times \text{均值}$**
- $P99.9 = 6.9 \times \text{均值}$

这个比值与利用率无关。但均值本身 = 服务时间 / (1-ρ)，随 ρ 爆炸：

利用率 ρ	P99 / 服务时间
0.3	6.6×
0.5	9.2×
0.7	15×
0.8	23×
0.9	46×
0.95	92×

这就是「×2 峰值余量」的理论依据，不是拍脑袋。若重排层跑到 90% 利用率，16ms 的服务时间对应 700ms 的 P99，直接击穿 200ms 预算。跑 50% 利用率则是 147ms，勉强在预算内。

收益递减：ρ 从 0.9 降到 0.5，机器翻倍，P99 改善 5 倍；从 0.5 降到 0.25，机器再翻倍，P99 只改善 1.5 倍。因此实操中很少把利用率压到 0.5 以下，越过这个点应改用其他手段。

9.3 对冲请求让慢的概率指数下降

同一分片同时发给 r 个副本、取最先返回的： $P(\text{全都慢}) = (1-p)^r$

副本数 r	慢的概率
1	1%
2	0.01%
3	0.0001%

两个公式恰好接上：9.1 要求单分片达到 P99.99，而 2 副本对冲正好把 P99 抬到 P99.99。

9.4 实操建议

超时截断是这里性价比最高的手段：设一个硬 deadline（如 80ms），超时的分片直接跳过，用剩余分片的结果返回。它把尾巴硬性锁死，不依赖任何分布假设，实现只是一个 deadline。

代价是被截掉的分片可能含最优商品，因此需要一个质量指标：**每次查询缺失了几个分片**。质量下降到什么程度可接受，是产品决策。

面试中的正确表述：「P99 我不敢在纸上算准，要压测；但我会用超时截断把尾巴锁死在预算内，而不是依赖单机延迟分布刚好达标。」

10. 设计要点回顾

- 1. **分片键决定架构**。按 doc_id 分片让 term→doc、doc→meta、doc→feature 三份数据共置，重排时零远程调用——省掉了整个特征服务这一跳。
- 2. **更新按代价分线，不按内容分线**。倒排改动贵但每秒几十次，feature 改动便宜但每秒数千次。混在一条线里会让贵的拖累便宜的。
- 3. **副本不摊薄写入**。更新量的除数是分片数，不是机器数。这是容量估算中最常见的错误之一。
- 4. **先检查假设是否荒谬**。「1% 的商品每秒变化」意味着目录每 100 秒翻新一次；改成合理速率后，问题自行消失。
- 5. **扇出的代价是百分位要求，不是吞吐**。N 个分片要求每个达到 $P99^{1/N}$ ，分片越多越苛刻。
- 6. **利用率是尾延迟的主要杠杆，且收益递减**。越过 50% 利用率之后，应改用对冲与超时而非堆机器。
- 7. **对后端面试官讲 ML 系统，把模型压成盒子**。讲清它周围的召回、特征、更新、扇出、成本；模型内部结构等对方追问再展开。